

Chapitre II

L'environnement Unix/Linux

(Suite)

Plan du Chapitre

1. Introduction aux systèmes « *Unix/Linux* »
2. Le système de fichiers
3. Sécurité du système de fichiers
4. **La programmation shell**
5. **Les filtres et les expressions régulières**
6. **Awk et Sed**
7. **Gestion des processus**

4. La programmation Shell

Le Shell : Présentation

Le Shell

- ✓ Interpréteur de commandes qui offre :
 - ☞ Langage de programmation : gestion de variables, structures etc.
 - ☞ Environnement (variables, sessions utilisateurs, historique, ...)
- ✓ Les + utilisés : ksh(korn shell), csh(c-shell), sh(Bourne shell), et **Bash (Bourne Again shell)**
- ✓ Exemple : fichiers d'environnement bash : `~/.bashrc`, `~/.bash_profile`, `~/.bash_history`, etc.

Variables d'environnement

✓ **\$ env** : liste les var. d'environnement.

☞ HOME : répertoire de travail de l'utilisateur de la session en cours

☞ PATH : les répertoires contenant des binaires.

☞ PWD : le répertoire en cours

☞ PS1 : valeur du prompt

☞ TERM

☞

✓ Leurs valeurs peuvent être personnalisées pour chaque utilisateur.

☞ En ligne de commande

☞ À travers les fichiers d'environnement génériques : /etc/profile , /etc/bashrc

☞ A travers les fichiers d'environnement des utilisateurs : ~/.bashrc, ~/.bash_profile

Caractères génériques et spéciaux

- ✓ ?
- ✓ *
- ✓ [..]
- ✓ {..}
- ✓ (..)
- ✓ &
- ✓ \
- ✓ |
- ✓ <, >
- ✓ \$
- ✓ ‘, ’, “
- ✓ \n
- ✓ etc.

Paramètres des commandes et des programmes

- ✓ 0 : nom de la commande
- ✓ \$: numéro du processus en cours
- ✓ @ : ensemble des paramètres
- ✓ # : nombre de paramètres
- ✓ ? : valeur de retour
 - ☞ 0 si succès
 - ☞ Code erreur sinon
- ✓ 1 : premier argument
- ✓ 2 : deuxième argument ...

Variables : déclaration et portée

- ✓ Variable accessible dans l'instance en cours du Shell uniquement :

`$ var=valeur`

`$ echo $var`

- ✓ Variable exportée vers toute instance ou fils du processus en cours :

`$ var = valeur`

`$ export var`

- ✓ Déclaration de constante : `readonly var`

Évaluation des variables(1)

- ✓ La commande **test** : évaluer une expression booléenne et renvoi la valeur vrai ou faux.
- ✓ Exemples :
 - ☞ Comparaisons numériques : ne ($\neq$), -gt(>), -ge($\geq$), -lt (<), -le ($\leq$), ...
 - **\$ test \$x -ne \$y**
 - ☞ Comparaison de chaînes de caractères :
 - -z \$ch: vrai si ch est de longueur nulle
 - -n \$ch: vrai si ch n'est pas nulle
 - Etc
- ✓ **\$ read var** : permet de lire à partir du clavier la valeur de var
- ✓ **\$ echo \$var** : permet d'afficher à l'écran la valeur de var

Évaluation des variables(2)

- ✓ Calcul d'expressions arithmétiques ou de chaînes de caractères :
expr

⇒ \$ expr \$1 : \$2 : compare les 2 chaînes \$1 et \$2, s'arrête dès que les chaînes diffèrent ou à la fin de \$1. Renvoie le nombre de caractères commun ou 0.

⇒ \$ expr \$a + \$b

⇒ \$ N=`expr \$N + 1`

⇒ \$ expr \$a * \$b

Les scripts Shell

- ✓ Le Shell offre un langage de scripts directement interprété par le shell.
- ✓ Pour exécuter un script shell dans Bash, il faut commencer le programme par la ligne :

`#!/bin/bash`

- ✓ Le script shell doit être un fichier exécutable (chmod +x ! avant l'exécution).
- ✓ Exécution : **`$ ./nom_script [arguments]`**
- ✓ Utilisez l'éditeur de texte gedit ou vi (voir manuel d'utilisation en annexe).
- ✓ Un script shell est un programme formé par des instructions utilisant des appels de commandes shell, des variables et des structures du langage shell.

La structure conditionnelle

```
if Condition
then
    instruction1...
    instructionN
elif
then
    ....
else
    ...
fi
```

Chaque ligne doit contenir un seul mot réservé et une seule instruction ou séparer les instructions par “;”

Condition : expression booléenne formulée par la commande test ou entre []. Exemples :

```
if test 3 -eq 3 ...
```

```
if [ 3 -eq 3 ]....
```

```
if test $1 == $2 ...
```

```
if [ $1 == $2 ]...
```

Les tests (Condition)

✓ Même syntaxe que la commande test

Tests sur types et permissions des fichiers :

- ✓ -r
- ✓ -w
- ✓ -x
- ✓ -b
- ✓ -c
- ✓ -d
- ✓ -f
- ✓ -s : vraie si taille de f sup. à 0.
- ✓ -e : vraie si f existe.

Tests sur des chaînes de caractères :

- ✓ -z ch: vrai si ch est de long nulle
- ✓ -n ch: vrai si ch n'est pas nulle

Tests numériques :

- ✓ -ne: not equal
- ✓ -le: less or equal
- ✓ -lt: less than
- ✓ -gt, -ge, ..

Opérateurs booléens :

- ✓ -o: or
- ✓ -a: and
- ✓ ...

La structure de choix

case \$chaîne **in**

Val1) Commandes... **;;**

Val2) Commandes... **;;**

...

esac

Exemple :

```
#!/bin/bash
```

```
echo -n "Entrez un nombre :"
```

```
read nombre
```

```
case $nombre in
```

```
1) echo "Vous avez tapé 1";;
```

```
2) echo "Vous avez tapé 2 ";;
```

```
*) echo "Vous n'avez tapé ni 1 ni 2";;
```

```
esac
```

Les boucles

```
for x in list
do
  commandes
done
```

List : ens. de valeurs

Exemple :

```
for i in 1 2 3 do
  echo $i
done
```

```
Until Test
do
  Commandes
done
```

```
While Test
do
  Commande
done
```

- **continue** : permet de passer à la prochaine itération d'une boucle
- **break** : permet de sortir d'une boucle

Les fonctions

- ✓ déclaration : `function nom { ... }`
- ✓ On accède à ses paramètres avec `$1...$n` (à ne pas confondre avec ceux du programme shell global !)

- ✓ Exp :
 1. `#!/bin/bash`
 2. **`function somme {`**
 3. `resultat = `expr $1 + $2``
 4. `echo $resultat`
 5. `}`
 6. `read x`
 7. `read y`
 8. **`somme $x $y # x sera $1 et y sera $2 lors de l'exécution de la fonction`**

Autres commandes du Shell

- ✓ Incrémentation : ++
- ✓ Décrémentation : --
- ✓ echo, read
- ✓ **exit** permet de terminer une commande ou un script en renvoyant un code de retour
- ✓ Les scripts shell :
 - ☞ Doivent être exécutable (+x).
 - ☞ Exécutés par : ./nom_script ou directement si leur chemin est inclus dans la variable PATH ou par : sh nom_script

Autres caractéristiques du shell Bash

✓ Les alias

☞ Créer des « raccourcis » vers des commandes.

☞ Exemple :

☞ alias dir = 'ls -l | grep ^d'

☞ unalias : annule l'effet d'un alias.

☞ Effet volatile (réservé à la session en cours) à moins qu'il soit déclaré dans /etc/bashrc ou ~/.bashrc.

✓ Gestion de l'historique :

☞ history

☞ !!

☞ !n ...

5. Filtres et commandes avancées du Shell

Les filtres (1)

✓ **sort**

☞ trie l'entrée ligne par ligne

☞ options: -r inverse l'ordre de tri

+n : ignore les n 1ers champs

-n : tri numérique

-u : afficher une instance unique de chaque ligne

-b : ignorer les blancs en début des champs

-f : confond les lettres majuscules et minuscules

Les filtres (2)

✓ **uniq**

☞ Affiche un fichier en détruisant les lignes consécutives dupliquées

☞ options: -u (affiche les lignes "uniques"),
 -d (affiche les lignes "dupliquées")

✓ **diff**

☞ compare deux fichiers

☞ options: -b (ignorer les lignes vides)

☞ exemple:

☞ diff fic1 fic2

Les filtres (3)

✓ cut

☞ sélectionne uniquement certaines colonnes du fichier passé en paramètre

☞ options:

☞ -f<liste> : liste des champs à afficher

☞ -c<liste> : liste des colonnes à afficher

☞ -d<char> : séparateur de champs

Exps :

☞ cut -d: -f1 /etc/passwd : affiche la première colonne du fichier /etc/passwd en considérant comme séparateur « : ».

☞ cut -c 10-16 /etc/passwd : affiche l'intervalle de caractères [10-16] de chaque ligne du fichier.

Les filtres (4)

✓ tr

☞ recopie l'entrée sur la sortie en substituant des caractères

☞ syntaxe: `tr [-cds] [s1 [s2]]`

☞ options:

☞ -c (complément de s1)

☞ -d efface les car. de s1

☞ -s tte séquence dans s1 est substituée par un car. unique dans s2

☞ exemples:

☞ `tr A-Z a-z < essai`

remplace les majuscules par des minuscules

☞ `tr A-Z a-z < essai | tr -sc a-z '\012'`

remplace les majuscules par des minuscules, puis remplace toute séquence de caractères non lettre minuscule par un retour chariot ('\012')

Les filtres (5)

✓ grep

☞ Recherche, dans le fichier passé en paramètre, les lignes vérifiant une expression régulière donnée

☞ syntaxe : `grep expr_reg [fichier]`

☞ exemples:

☞ `grep 'toto' essai`

cherche dans essai toutes les lignes qui contiennent le mot toto

☞ `grep 'toto$' essai`

cherche dans essai toutes les lignes qui se terminent par le mot toto

☞ `grep '^[A-Z]' essai`

cherche dans essai toutes les lignes qui commencent par une majuscule

Les filtres (6)

✓ grep : Options :

- ☞ -w mot : recherche le mot exact
- ☞ -v expr : recherche les lignes qui ne contiennent pas expr
- ☞ -c : affiche le nombre de lignes résultats
- ☞ -f fichier : lit une expr. régulière à partir de fichier
- ☞ -B n : affiche n lignes avant chaque ligne trouvée
- ☞ -A n : affiche n lignes après chaque ligne trouvée
- ☞ -C n : affiche n lignes avant et après celle recherchée
- ☞ -i : confondre min. et maj.

Les expressions régulières (1)

- ✓ `[..]` un des caractères, par énumération ou plage

☞ Exemples:

☞ `^[aA]`: tous les fichiers commençant par a et A

☞ `^[a-f]`: tous les fichiers commençant par a, b, c, d, e et f

- ✓ `{..}` ensemble de valeurs
- ✓ `(..)` grouper une sous expression régulière afin de la ré-appeler après par `\1`, `\2` etc
- ✓ `\` annule la sémantique d'un caractère spécial
- ✓ `[^...]` sauf les caractères définis

☞ Exemple:

☞ `[a-m^i]`: toutes les lignes contenant une lettre de a à m sauf i

- ✓ `#` commentaire

Exemples

- ✓ Rechercher toutes les occurrences des mots «terre» et «ciel» dans le fichier roman.txt avec les numéros des lignes.
- ✓ Rechercher toutes les lignes qui commencent par a ou A.
- ✓ Rechercher les lignes contenant un chiffre
- ✓ Chercher toutes les lignes ne finissant pas par un signe de ponctuation (point, virgule, point-virgule, deux-points, point d'interrogation, point d'exclamation)

Solutions

- ✓ `grep -inw '\(ciel\ | terre\)' fichier`
- ✓ `grep '^[aA]' fichier`
- ✓ `grep '[0-9]' fichier`
- ✓ `grep -v '[.,;:?!]$\ ' fichier`

Recherche de fichiers (1)

find chemin [option] expression

✓ Options:

 -name motif

 -user nom

 -atime n : non utilisé depuis moins de n jours

 -mtime n : non modifié depuis moins de n jours

 -newer fichier : plus récent (date de création) que fichier

 -exec commande

 -inum n

 -type x

 b

 c

 f

 d



Exemples

- ✓ Appliquer `ls -l` sur tous les fichiers source `c` ou objet `o` et modifié il ya moins de 30 jours.
- ✓ On recherche des fichiers dont la taille est supérieure à 30 K
- ✓ Chercher tous les fichiers dont les noms commencent par `a` ou `A`, suivi d'éventuellement quelques lettres ou chiffres, et se terminent par un chiffre entre 3 et 6 ?
- ✓ Comment fait-on pour indiquer que le fichier recherché a été modifié il y a plus de 30 jours ? Il y a 30 jours ? Il y a moins de 30 jours?

- ✓ `find / \( -name *.c -o -name *.o \) -mtime 30 -exec ls -l {} \;`
- ✓ `find / -type f -size +30k -print`
- ✓ `find . -name '[aA]*'[3-6]`
- ✓
 - `-mtime 30` : il y a 30 jours;
 - `-mtime +30` : il y a 30 jours ou plus;
 - `-mtime -30` : il y a 30 jours ou moins.

Autres commandes

- ✓ wc : compter le nombre de lignes(-l), le nombre de mots(-w) et le nombre de caractères (-c) d'un fichier.
- ✓ merge, paste, ...
- ✓ Compression de fichier : gzip, bzip, compress, ..
- ✓ Décompression de fichiers : gunzip, bunzip, uncompress, ..
- ✓ Archivage : tar
- ✓ Montage / démontage de partitions ou de volumes disques : mount / umount
- ✓ ...

6. Awk et Sed

AWK

- ✓ Outil programmable de transformation de texte
- ✓ Syntaxe proche du langage C
- ✓ Calculs de base, fonctions, ...
- ✓ Intègre la recherche des expressions régulières

`$ awk 'programme-awk' [variable=valeur] fichiers ...`

ou

`awk -f fichier-awk [variable=valeur] fichiers ...`

✓ Options

- ☞ `-f fichier-awk` : charge le programme ou les commandes awk à partir d'un fichier (pratique si plusieurs lignes)
- ☞ `-F c` : **définit** le séparateur de champs du fichier en entrée au caractère `c`

AWK

✓ Structure d'un programme awk

Corps {
 BEGIN {action0}
 regex1 {action1}
 regex2 {action2}

 END {actionF}

✓ Principe

✓ **Action 0** : Initialisation + effectuer action0 avant la lecture de la 1ère ligne du fichier donné en paramètre

✓ **Corps** : exécuté pour chaque ligne du fichier en entrée

☞ si regex1 est vérifiée effectuer action1

☞ si regex2 est vérifiée effectuer action2

☞ etc ...

✓ **Action F** : exécutée après le traitement de la dernière ligne : Terminaison + effectuer actionF

✓ Si « regex » omise : action toujours effectuée pour chaque ligne du fichier

AWK : les variables prédéfinies(1)

- ✓ Pour le « *programme-awk* »
- ✓ Chaque **ligne** ("**Record**") est automatiquement séparée en **champs** ("**Fields**") avec comme séparateur par défaut: blancs et/ou tabulations.
 - ☞ **NR** : numéro de la ligne courante (Record)
 - ☞ **NF** : nombre de champs de la ligne courante (Fields)
 - ☞ **\$0** contenu de la ligne courante
 - ☞ **\$1, \$2 ... \$NF** contenu du 1er ... dernier champ
 - ☞ **RS** : séparateur de lignes (par défaut = \n)
 - ☞ **FS** : Séparateur de champs (par défaut = blanc et tab)
 - ☞ **ORS, OFS** séparateurs de ligne et de champs pour l'affichage en sortie
- ✓ **Remarque** : La variable **FS** peut aussi être initialisée lors de l'appel de **awk** via l'option: -F

AWK : les variables AWK(2)

✓ Exemples :

☞ Afficher les noms des utilisateurs connectés et l'instant de leur connexion :

```
$ who | awk '{ print $1, $5 }'
```

☞ Afficher la 3ème et les deux dernières colonnes du fichier annuaire :

```
$ awk '{print $3 " " ($(NF-1) + $NF)}' annuaire
```

☞ Afficher un fichier avec le num. de chaque ligne sur 4 caractères

```
$ awk '{printf "%4d%s\n", NR, $0}' fichier
```

AWK : Structures de contrôle

- ✓ Syntaxe proche du langage C
- ✓ **if** (condition) instructions [**else** instruction]
- ✓ **while** (condition) instructions
- ✓ **for** (expression; condition; expression) instructions
- ✓ **break, continue, exit**
- ✓ Remarques :
 - ☞ Variables utilisateur et tableaux définis implicitement
 - ☞ Opérateurs de comparaison et de calcul du langage C
 - ☞ Fonctions d'impression et structures de contrôle similaires au langage C

AWK : Fonctions prédéfinies

✓ Chaînes de caractères

- 👉 **index** (s1, s2) : retourne la position de s2 dans s1
- 👉 **substr** (s, c [, l]) : retourne la sous chaîne de s qui débute à c et de longueur l sinon jusqu'à la fin de s.
- 👉 **getline()** : lit la ligne suivante, retourne 1 si la fin de fichier sinon 0.
- 👉 **print** [expr1 [expr2]]
- 👉 **printf** (format, expr1, expr2 ...)
- 👉 **length** (chaine)
- 👉 ...

✓ Arithmétiques

- 👉 **sqrt** (x)
- 👉 **log** (x)
- 👉 **exp** (x)
- 👉 **Sin**(x)
- 👉 **Cos**(x)
- 👉 **int** (x)
- 👉 ...

AWK : Exemple 1

- ✓ Soit le fichier de données '*population*' :

Canada	3852	25	Amerique
Chine	3705	1032	Asie
France	211	55	Europe

- ✓ Écrire un script '*total*' qui transforme le fichier ainsi :

<u>Pays</u>	<u>Superf</u>	<u>Pop</u>	<u>Cont</u>
Canada	3852	25	Amerique
Chine	3705	1032	Asie
France	211	55	Europe
<u>TOTAL</u>	<u>16417</u>	<u>1387</u>	

- ✓ Écrire le programme '*maxpop*' qui permet de donner le pays ayant la plus grande population

AWK : Exemple 1

Programme total

```
BEGIN {printf("%10s %6s %5s %s\n",  
"Pays","Superf","Pop","Cont")  
super=0  
pop=0}  
{  
printf ("%10s %6s %5s %s\n", $1, $2,  
$3, $4)  
superf = superf + $2  
pop = pop + $3  
}  
END {printf("\n %10s %6s %5s\n",  
"TOTAL", superf, pop)}
```

✓ Programme maxpop

- ✓ BEGIN {maxpop=0}
 - ✓ {
 - ✓ if (maxpop < \$3) {
 - ✓ maxpop = \$3
 - ✓ pays = \$1
 - ✓ }
 - ✓ }
 - ✓ END { print "Pays: " pays "/
Max-Pop: " maxpop }
- ✓ Résultat
Pays: Chine / Max-Pop: 1032

AWK : Exemple2

✓ Utilisation des tableaux

👉 Exemple : Compter les apparitions de mots dans un fichier

```
awk '
{ for (i=1; i<NF; i++) num[$i]++ }
END { for (mot in num) print mot, num[mot] }' fichier
```

Tableau de taille dynamique : cases allouées selon les mots trouvés

- ✓ Indexes du tableau : mots
- ✓ Contenu des cases : entiers

SED: éditeur de flots

- ✓ Permet de gérer des flots de données en exécutant des opérations de substitution.
- ✓ Utilise un tampon dans le quel elle transfère ligne par ligne le contenu d'un fichier pour les traiter.
- ✓ Syntaxe :

`sed [-n] [requête][-e requête]* [-f script] [fichier]*`

- ◆ -n : n'affiche en sortie que les lignes sur les quelles on applique les requêtes
- ◆ -e : doit précéder chaque requête s'il y a plusieurs
- ✓ Format d'une requête :

`[adresse1[, adresse2]] commande [arguments]`

- ◆ adresse :
 - ◆ Vide : traite toute chaîne
 - ◆ num. absolu d'une ligne de texte
 - ◆ \$: dernière ligne traitée
- ◆ Commande : expression régulière ou commande sed

SED: éditeur de flots

- ✓ Les commandes sed
 - ♦ a\ ch :ajoute ch au début du flot de donnée
 - ♦ b etiq : saut à la commande etiq
 - ♦ c\ remplace des lignes
 - ♦ d détruit la ligne courante et lit la suivante
 - ♦ i\ insère le texte
 - ♦ l imprime la ligne même les caractères non imprimables
 - ♦ p imprime la ligne
 - ♦ s/anc/nouv/f substitue l'expression régulière anc par la chaîne nouv avec comme valeurs de f :
 - ♦ g : remplace toutes les occurrences
 - ♦ p : écrire le résultat si une substitution est réalisée
 - ♦ w fichier : enregistrer dans fichier si une substitution est réalisée

Sed : Exemples

- ✓ Remplacer un espace suivi par un nbr quelconque de caractères par un seul espace.

```
$ who | sed 's/ .*/ / '
```

- ✓ Afficher toute les lignes d'un fichier jusqu'au mot 'fin'.

```
$ sed '/fin/q' fichier
```

- ✓ Remplacer toutes les fins de ligne par le caractère ':'

```
$ sed 's/\n/:/g' fichier
```

7. Gestion des processus

Les processus

- ✓ Processus = objet dynamique qui représente une instance d'un programme en cours d'exécution et son contexte
- ✓ Caractéristiques:
 - ☞ identification (pid)
 - ☞ identification du proc. parent (ppid)
 - ☞ propriétaire
 - ☞ priorité
 - ☞ Pourcentage d'utilisation de la mémoire
 - ☞ ...
- ✓ Pour voir les processus en cours: **ps, pstree, top, ...**

Les processus

✓ Infos retournées par **ps**:

[lewandow:~] ps					
numéro de processus		terminal associé	état du processus:	temps CPU utilisé	commande exécutée
PID	TT	STAT	TIME	COMMAND	
3899	p1	S	0:00.08	-zsh	
4743	p1	S+	0:00.14	emacs	
4180	std	S	0:00.04	-zsh	

R actif

T bloqué

P en attente de page

D en attente de disque

S endormi

Z tué

Les processus

✓ Options de ps:

- A liste tous les processus actifs du système
- ef liste tous les processus actifs du système avec plus d'informations
- u *user* affiche les processus d'un utilisateur donné
- l : affiche en plus les infos de priorités

✓ Exécution en avant plan != exécution en arrière plan

✓ Processus en arrière-plan: &

(le terminal n'est pas bloqué)

exemple: **emacs monfichier.c &**

Les processus

- ☞ jobs : liste des processus lancés en arrière plan
- ☞ fg %n : ramène un processus en avant plan
- ☞ bg %n : ramène un processus suspendu en arrière plan
- ✓ Lancer une cde en différé : exécution indépendante de la session et du shell en cours :
 - ☞ nohup cde
- ✓ Envoi de signaux : **kill**
 - ☞ kill -l : liste des signaux
 - ☞ Raccourcis clavier:
 - ☞ Ctrl + c : arrêt
 - ☞ Ctrl + z : suspendre
 - ☞ Exemple : Tuer un processus immédiatement :
\$ kill -9 <PID>

Les processus

✓ Les priorités :

☞ [-19..20] : du plus prioritaire au moins prioritaire. Exp: les processus root sont par défaut de priorité 0, ceux des utilisateurs non privilégiés sont 10

☞ **nice** : lance une commande avec une certaine priorité

↳ nice +n | -n cde

☞ **renice** : change la priorité d'un processus en cours d'exécution.

↳ renice +n | -n pid